
CS780: Deep Reinforcement Learning

Capstone Project

OBELIX: The Warehouse Robot

Khushal Wadhwa
Roll No.: 230559
kwadhwa23@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

This work addresses the OBELIX warehouse robot task, where an agent must learn to locate, attach to, and push a box under partial observability. A Duelling Double Deep Q-Network (D3QN) with group-based prioritized experience replay was employed to handle perceptual aliasing and sparse rewards. Curriculum learning, potential-based reward shaping, and action space simplification were incorporated to stabilize training. Experiments across varying difficulty levels demonstrate the challenges posed by occlusion, blinking, and moving targets. A critical implementation bug affecting replay buffer integrity was identified and resolved, enabling meaningful learning and improved policy behavior.

1 Competition Result

Codabench Username: k_230559

Final leaderboard rank (Test Phase): 89

Leaderboard rank – Level 1 (static): 76

Leaderboard rank – Level 2 (blinking): 50

Leaderboard rank – Level 3 (moving + blinking): 43

Number of valid submissions made (Development(Level 1 , Level 2, Level 3) and Test):

Phase	Number of Submissions
Level 1	8
Level 2	3
Level 3	2
Test Phase	5
Total	18

Table 1: Number of Valid Submissions

2 Introduction and Problem Description

In robotics, designing control policies using traditional rule-based systems often fails when the agent must act based on dynamic, incomplete and ambiguous sensor information. Reinforcement Learning provides a framework to address this challenge using adaptive decision-making, enabling the agent to learn action policies directly from interaction, optimizing long-term reward without requiring explicit state estimation or hand-crafted heuristics. The OBELIX task serves as an instance of such a setting, where the agent must operate under partial observability and learn to perform a multi-stage manipulation task using limited sensor feedback.

In this project, the environment used is a simulation of the OBELIX robot, originally introduced by Mahadevan and Connell (1991). The task models a simplified warehouse scenario in which a mobile robot must locate, attach to, and push a box to the boundary. The robot operates in a 2D arena and is restricted to a discrete action space consisting of 5 actions: 1 forward motion, and 4 in-place rotations (left/right by 22° and 45°). The environment is partially observable, which means the robot does not have access to the full state of the world. Instead, its perception is based on a set of binary sensors, resulting in an 18-dimensional observation vector:

- 16 bits from 8 sonar sensors (each providing near and far detection),
- 1 infrared (IR) sensor indicating close frontal contact,
- 1 stuck flag indicating whether the robot is blocked or unable to move.

This makes the problem a Partially Observable Markov Decision Process (POMDP), significantly increasing its difficulty. This limited sensing leads to a large state space (on the order of 2^{18} possible observations) and introduces perceptual aliasing, where identical observations may correspond to different underlying configurations.

As stated in the original work itself, the task consists of three implicit sub-tasks that must be learned for successful task completion:

1. Finding the box using directional sensor cues,
2. Pushing the box while maintaining alignment,
3. Unwedging when the robot becomes stuck against walls or obstacles.

The simulated environment is evaluated across three difficulty levels:

- Level 1 (Static Box): The box remains stationary.
- Level 2 (Blinking Box): The box intermittently disappears, making perception unreliable.
- Level 3 (Moving + Blinking Box): The box moves and disappears, introducing non-stationarity.

Additionally, wall obstacles may be present, requiring the robot to navigate through narrow openings while pushing the box.

The reward structure comprises of sparse yet structured signals derived from sensor activations and task progress. At each time step, a penalty of -1 is applied to encourage efficient behaviour. Small positive rewards are provided only upon the first activation of sensor signals within an episode, ranging from +1 to +5, depending on the sensor direction and whether the near or far bit is triggered. A reward of +100 is given upon successful initial attachment to the box, while a heavy penalty of -200 is applied when the agent becomes stuck against walls or the boundary. Despite these intermediate signals, the reward remains sparse and indirect, making it difficult for the agent to infer long-term strategies.

The episode terminates when the robot successfully attaches to the box and pushes it to the boundary, at which point a large terminal reward of +2000 is provided.

Overall, the environment combines high-dimensional discrete observations, partial observability, delayed rewards, and multi-stage objectives, making it a challenging benchmark for reinforcement learning.

ment learning algorithms. However, with the integration of deep architecture, the automation problem can be re-imagined.

The objective: design a policy that maximizes mean cumulative reward across all difficulty levels, while being robust to randomness and environmental variations. Ultimately, the agent must consistently complete the task of pushing the box to the boundary.

3 Approach and Model Evolution

3.1 Choice of Algorithm

The problem was initially approached using tabular reinforcement learning, specifically Q-Learning and $Q(\lambda)$. Given the discrete 18-bit observation space, tabular methods were sufficient in principle. However, empirical results revealed significant perceptual aliasing: identical observation vectors often corresponded to distinct physical configurations. Tabular methods failed to generalise across these semantically similar states, as they treat each observation as an independent state and do not generalise across similar observations, leading to poor sample efficiency and noisy value estimates.

To address these limitations, the DQN family of algorithms was chosen as a suitable alternative, as value-based methods are well-suited to the discrete set of action primitives, and actually informative transitions are sparse, which makes off-policy learning with experience replay particularly effective for improving sample efficiency. Ultimately, a Duelling Double Deep Q-Network (D3QN) architecture was finalised. This optimizes learning through three key mechanisms: neural function approximation to resolve perceptual aliasing via shared state representations; Double DQN to mitigate overestimation bias by decoupling action selection from evaluation; and a Duelling architecture that decomposes the Q-function into state-value $V(s)$ and advantage $A(s, a)$ components.

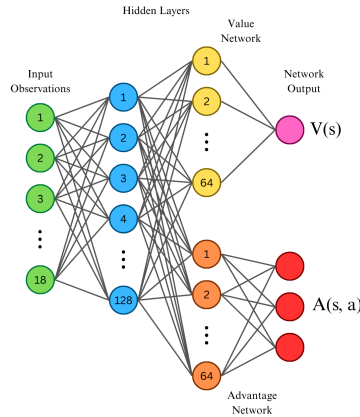


Figure 1: Duelling Double DQN architecture used in this work.

The Q-values are computed as: $Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right)$

3.2 Exploration Strategies and Handling Sparse Rewards

The central challenge of this environment is the presence of sparse and delayed rewards. Several state-of-the-art strategies were evaluated to enhance learning to find the one most suitable for our task:

Hindsight Experience Replay (HER): Learns from failure by relabeling transitions with achieved states as the goals. However, it proved inapplicable because the observation space

could not be changed to a goal-conditioned MDP formulation.

Random Network Distillation (RND): Incentivises exploration via intrinsic rewards based on prediction errors of a fixed random network. It was observed to bias the agent towards novel but task-irrelevant sensor configurations, degrading objective performance.

Noisy Networks for Exploration: Injects trainable stochastic noise into the linear layers to induce state-dependent exploration; however, the learned noise parameters (σ) decayed prematurely, suppressing exploration even before the policy could reach convergence.

Final Configuration:

Group-Based Prioritized Experience Replay (PER): Regular PER can suffer from redundant sampling, where identical observations that only differ by minor sensor noise dominate the replay buffer. This causes the agent to train on common, repetitive situations while ignoring the rare, critical ones.

To address this, a group-based strategy was introduced. Instead of treating each observation independently, experiences are clustered based on a state abstraction function. This function simplifies them into a lower-dimensional representation by focusing on the direction of the sensors. By sampling from these groups, it is ensured that the agent sees a diverse range of scenarios, preventing the learning process from getting bogged down in minor sensor fluctuations. Within each group, prioritization is still maintained using TD-error, combining the benefits of PER with improved diversity in sampling.

Decaying ϵ -Greedy: For exploration, ϵ -Greedy proved to be the most reliable. Exploration was initialized at 100% ($\epsilon = 1.0$) and decayed linearly to a terminal value of 10% ($\epsilon = 0.1$) over 100,000 steps. This exploration floor was maintained for the rest of the training to ensure the agent retained a baseline level of stochasticity in the sparse-reward environment.

3.3 Reward Shaping

While a terminal reward is provided upon successfully pushing the box to the boundary, intermediate feedback is limited, making it difficult for the agent to learn meaningful behaviours early in training. To address this, several reward shaping strategies were explored.

Geometric Distance-Based Shaping: Initial rewards were based on the Euclidean distance reduction between the robot, the box, and the boundary by accessing their locations through the environment’s internal code. This failed as local rotations and movements in any possible orientation generated noisy signals, reinforcing inefficient behaviours.

Encouraging Motion: To counter degenerate policies like spinning, simple reward modifications were attempted, such as removing penalties for forward movement or high penalties for overly repeated actions. However, these naive modifications led to more degenerate policies, such as continuously moving forward without regard to task objectives. These approaches were therefore discarded, as they failed to encode task-relevant structure.

Projection-Based Progress: Rewards were reformulated based on the displacement vector’s projection onto the goal direction. While more aligned with task objectives, this introduced high variance due to the robot’s random initial orientations, leading to training instability.

3.4 Final Adaptations From the Original Paper

Potential-Based Reward Inspired by Weighted Hamming Distance: To ensure policy invariance and consistency, we adopted a PBRs approach based on sensor-derived potentials. Instead of location-based rewards, a potential function $\Phi(s)$ is defined by partitioning it into semantically distinct sensor groups. The modified reward $r' = r + \gamma \cdot (\Phi(s') - \Phi(s))$.

Specifically, the potential was defined as:

$$\Phi(s) = 3 \cdot [\text{forward}] - ([\text{left}] + [\text{right}]) + 5 \cdot [\text{IR}] - 10 \cdot [\text{stuck}]$$

This formulation successfully incentivized task-relevant motion without the artifacts of raw geometric shaping. In addition, a reward is also included for the number of state changes. This encourages the agent to keep moving and actively keep changing its sensor readings.

State Space Simplification: For the purpose of experience replay and state grouping, the 18-bit observation was compressed into a 5-feature semantic abstraction:

1. Forward, 2. Left, 3. Right Detection, 4. Infrared signal, 5. Stuck Condition

This compact representation allows the agent to prioritize diverse environmental interactions, ensuring that the replay buffer remains informative throughout all phases of the task.

Task Decomposition: To manage the distinct requirements of searching and pushing along with obstacle avoidance, a "FinderEnv" sub-environment was created, which isolated the location and approach phases. The first 2000 episodes were trained on the **FinderEnv**. This was subsequently followed by 2000 episodes in the complete environment.

4 Experiments and Implementation Details

4.1 Training Setup

All experiments were conducted using the default environment configuration, with the box speed, scaling factor, and arena size left unchanged to ensure consistency with the evaluation settings. Training was performed on a 13th-generation Intel i7 CPU, using a curriculum-based approach to progressively increase task difficulty while stabilizing learning. The curriculum was structured as follows:

Phase	Episodes	Difficulty	Wall Obstacles
Phase 1	2000	0	False
Phase 2	500	3	False
Phase 3	1400	3	True
Phase 4	100	3	False

Table 2: Curriculum training schedule

The initial phase focuses on low-difficulty settings without obstacles to enable the agent to reliably learn exploration and box localization, which was observed to be the most challenging component of the task. Once this behaviour is acquired, subsequent phases introduce higher difficulty and environmental constraints, allowing the agent to incrementally learn pushing and obstacle navigation behaviours.

Training was primarily conducted using the **FinderEnv** environment with a maximum horizon of 1000 steps per episode, while the full **OBELIX** environment used a horizon of 2000 steps to accommodate longer trajectories required for task completion.

4.2 Hyperparameters

The model was trained using a Duelling Double DQN framework with Prioritized Experience Replay. The final hyperparameters are summarised in Table 3.

An ϵ -greedy exploration strategy was employed with: $\epsilon_{\text{start}} = 1.0$, $\epsilon_{\text{end}} = 0.1$

The decay was performed over 100,000 steps. This ensures that each phase of the curriculum

Parameter	Value
Discount factor (γ)	0.99
Learning rate (lr)	1×10^{-4}
Replay buffer size	10^5
Replay buffer sample size	256
Polyak averaging coefficient (τ)	0.01
PER: alpha (α)	0.6
PER: beta (β)	0.4
Training frequency	Every 4 steps
Target update frequency	Every 4 steps

Table 3: Training hyperparameters

begins with high exploration, allowing the agent to adapt to new dynamics, followed by sustained exploration at a 10% level to prevent premature convergence.

In the group-based PER mechanism, a maximum of 64 samples per group was enforced during sampling. This approach ensures balanced learning across different behavioural contexts while also retaining prioritization based on TD-error.

4.3 Action Space Simplification

The original action space of five actions was reduced to three: {L22, FW, R22}

This simplification significantly reduces the exploration space and improves learning efficiency, while still retaining sufficient control for navigation and alignment. Empirically, finer rotational granularity and additional actions were found to increase exploration complexity without providing meaningful gains in performance.

4.4 Policy Augmentation

During evaluation, a small heuristic was incorporated to improve robustness in a major failure case. Normally, when the agent enters a stuck state, the policy makes it spin in place. So, whenever indicated by the corresponding stuck observation flag, a predefined sequence of rotation and forward movement is executed to recover from the configuration. Outside of this condition, actions are always selected greedily based on the predicted Q-values. Additionally, a 5% chance of random action selection is retained during inference to mitigate any other deterministic failure modes. The heuristic is triggered only under the stuck condition and does not otherwise interfere with standard policy execution.

4.5 Seeding

All experiments were conducted using a fixed seed (seed = 0). Multiple seeds were not explicitly used, as the environment itself introduces significant stochasticity through randomized initial conditions, object dynamics, and sensor noise. This inherent variability provides sufficient diversity in training trajectories, reducing the marginal benefit of additional random seeds while maintaining computational efficiency.

5 Results

During the training phases (Levels 1-3), the agent always converged to a spinning policy in which the agent rotated in place without making meaningful progress toward the task objective. Although the evaluation scores obtained were reasonably strong, I believe that this does not

reflect the core objective of the task, which is learning a policy for the successful completion of the box-pushing behaviour. As a result, reporting the performance for these phases was not considered informative, since the policies did not correspond to meaningful or successful solutions.

Table 4 presents the performance of the final trained agent in the test phase across different difficulty levels and environmental configurations. Results are as per the Codabench leaderboard parameters of cumulative reward and standard deviation.

Wall	Difficulty	Cumulative Reward	Std Deviation
True	1	-15781.8	10717.591
True	2	-26811.3	22630.986
True	3	-9626.1	11422.878
False	1	-7857.9	10740.642
False	2	-6944.8	8407.613
False	3	-5773.7	7480.704

Table 4: Final test phase performance across different difficulty levels and wall configurations

The results show a clear difference in performance between environments with and without obstacles. In the absence of walls, the agent achieves significantly higher cumulative rewards across all difficulty levels. This is mostly due to the increased number of collisions of the agent with the wall, which is causing it to accumulate a lot more negative reward when walls are present.

Interestingly, performance does not degrade monotonically with difficulty. Although difficulty level 2 with walls exhibits the lowest performance, level 3 shows comparatively improved results. A reason for this is that the agent has learnt a good structural pattern for level 3: it differentiates between the wall and the object based on the fact that the wall remains stationary, but the object moves around. Using this, the agent can locate the object much better.

The high standard deviation observed across several settings indicates significant variability in agent performance. This is expected in a partially observable and stochastic environment, where small differences in initial conditions can lead to divergent trajectories. The relatively lower variance in no-wall settings further supports the conclusion that obstacle interactions are a major source of instability. It is also an indicator of the agent actually trying to formulate a policy

6 Error Analysis and Discussion

Failure Modes:

- **Boundary-induced spinning:** When the agent reaches the boundary away from the box, the observation collapses to a near-empty state (typically all zeros except the boundary bit). The agent learns that forward motion often leads to such stuck states and converges to a policy of rotating in place, resulting in a local minimum. A heuristic was introduced to mitigate this behavior.
- **Occlusion by walls (Levels 1 & 2):** When the agent and box are on opposite sides of a wall, the agent cannot distinguish the object from the wall due to limited observations. This makes the box effectively undetectable, leading to guaranteed failure in such configurations.
- **Blinking-induced tracking failures (Level 2):** The intermittent visibility of the box causes the agent to lose track due to false negatives. Since the policy lacks temporal memory, it cannot maintain a belief of the object’s position during periods of invisibility.

- **Reactive oscillation near walls (Level 3):** When a wall appears in the front sensors, the agent alternates between left and right rotations to check for motion. If the obstacle moves, it resumes tracking; otherwise, it continues oscillating, leading to inefficient behavior.
- **Delayed or failed interception (Level 3):** The agent reacts to the current position of the moving box rather than predicting its trajectory, resulting in chasing behavior instead of effective interception.

Limitations and Future Improvements

- Incorporate temporal memory (RNNs or LSTMs) to handle partial observability.
- Improve observation representation to better distinguish walls from objects.
- Introduce predictive modelling for moving targets.
- Refine reward design to encourage better generalized learning.
- Use more elaborate curriculum learning to improve robustness across difficulty levels.

7 Critical Implementation Bug in Environment (Root Cause of Non-Convergence)

A major issue preventing learning was traced to how the environment handled observations. The observation state s was being returned by reference rather than by value. Since the environment reused the same NumPy array for subsequent observations, all stored experiences in the replay buffer pointed to the same underlying memory. As NumPy arrays are mutable, this caused previously stored states to be unintentionally modified over time. Consequently, the replay buffer contained corrupted transitions, making it impossible for the agent to learn a stable policy. This issue was identified during late-stage debugging and resolved by explicitly storing copies of the states (using `s.copy()` and `s2.copy()`) in the replay buffer.

8 Conclusion

This project explored Reinforcement Learning in a highly challenging partially observable robotic setting, where the agent must learn multi-stage behaviours including search, alignment, and object manipulation. Using a Duelling Double DQN framework with group-based prioritized replay, curriculum learning, and potential-based reward shaping, the agent was able to develop some structure in its behaviours across increasing difficulty levels. While performance remained sensitive to environmental complexity, especially in the presence of walls, meaningful improvements over degenerate policies were achieved.

A key takeaway from this work is that reinforcement learning in POMDP settings is highly sensitive to representation, exploration, and implementation details. Even subtle issues, such as incorrect handling of state memory, can completely prevent convergence. Additionally, agents tend to exploit shortcuts in sparse reward settings, emphasizing the importance of careful reward design and training strategies.

Future work could focus on using improved computational power to incorporate bigger, better architectures like temporal memory (e.g., LSTM layers) to better handle partial observability, developing predictive models for moving targets, and improving state representations to distinguish between objects and obstacles. Learning to correctly differentiate between the wall and the objects is one of the most important things that must be taught to the agent. Extending this framework to more complex environments, including multiple objects or real-world robotic systems, would further evaluate the robustness and generalization of the approach.

LLM Usage Declaration

- Which Large Language Model (LLM) did you use to complete the competition or write the report?
ChatGPT & Claude
- In which part did you use the LLM, and how did it help you?

Part	Used LLM? (Yes/No)	Purpose of Using LLM
Understanding concepts	Yes	Understanding if the concepts that I wanted to use were truly suitable to work in my case.
Ideation / brainstorming	Yes	Finding research papers online to implement novel methods, and to understand how they work and how I could use it to implement in my case.
Debugging small code snippets	Yes	Help with all the syntax-related errors along with asking LLMs to suggest every possible way to speed up training.
Plotting / visualization code	No	-
Grammar / writing style	Yes	Help with paraphrasing of self-written content to give it a formal flavour in order to make it suitable for a research paper-style report.
Report structure / section ideas	No	Entire report structure and sectional breakdown was done by me, and by referring to the template.
Understanding Code	Yes	Used LLM to understand files like obelix.py, evaluate.py, or even the starter code provided.
Ancillary code generation	Yes	generating ancillary parts of the code like logging and other debug prints like counting different states in replay buffer.

- Did you refer to any other sources or websites apart from the LLM and lecture slides?
If yes, please mention clearly (papers, blogs, GitHub repos, textbooks etc.)
Whatever resources I have referred to have been mentioned in the citations below

- Apart from any LLM usage, what was your own contribution?
I have only used LLMs for concept clarification, syntax help, and help with ancillary code, coming up with the grouping idea based on sensor direction, the final reward shaping method, these kinds of things were all my own contribution. Deciding how to use and what to use, and why it is ultimately not working and why I will have to discard them, all these things were essentially done by me.

References

1. S. Mahadevan and J. Connell, "Automatic Programming of Behavior-Based Robots using Reinforcement Learning," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 1991.
2. M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, and W. Zaremba, "Hindsight Experience Replay," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
3. Y. Burda, H. Edwards, D. Pathak, A. Storkey, T. Darrell, and A. A. Efros, "Exploration by Random Network Distillation," in *International Conference on Learning Representations (ICLR)*, 2019.
4. M. Fortunato, M. G. Azar, B. Piot, J. Menick, I. Osband, A. Graves, V. Mnih, R. Munos, O. Pietquin, C. Blundell, and S. Legg, "Noisy Networks for Exploration," *arXiv preprint arXiv:1706.10295*, 2017.